LMU LUDWIG-MAXIMILIANS-UNIVERSITÄT MÜNCHEN

Advanced Topics in Parallel Computing

Dr. Karl Fürlinger
Lehrstuhl für Kommunikationssysteme und Systemprogrammierung

Blocking Concurrency

Implementing Concurrent Objects with Locks



Overview

- Recap: Options for implementing concurrent objects:
  - **Blocking** – aka. **lock-based implementation**
  - **Non-blocking** – **lock-free** or **wait-free** progress guarantees
- A blocking implementation is often the simplest and best solution
  - Despite the conceptual appeal of LF/WF approaches
- Advantages of blocking synchronization :
  - Relative implementation **simplicity**
  - Performance may be **good enough** for a particular use case (e.g., when contention is low)
- Disadvantages:
  - A delayed thread may delay all others
  - Danger of deadlocks when multiple locks are used

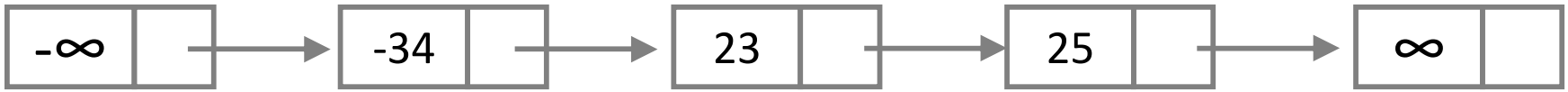
Using Locks

- Several options in terms of how locks may be used
  - **Coarse-grained locking**: one lock per object, acquire it for each method call
  - **Fine-grained locking**: lock smaller parts of the data structure, so that more operations (on different parts of the DS) can happen at the same time
  - **Optimistic schemes**: reduce the frequency of locking by preparing to perform operations without locks; then, while holding locks, validate that operations are still safe before actually performing them
  - **Lazy schemes:** Perform some expensive operations lazily (e.g., first logically, then physically delete a node), such that other operations may proceed concurrently

Use case: Set Realized as a Sorted List

- Goal: Implement a **Set<T>** abstraction
  - Set abstraction: a value is either present or not (**no duplicates**)
  - **bool add(T val)** – add val, return *true* if val was not present before
  - **bool remove(T val)** – remove val, return *true* if val was present
  - **bool contains(T val)** – return *true* if val is present

- Idea: represent the set as **a sorted linked list**
  - We assume that there is a total order on T and values of type T can be efficiently compared, e.g. integers
  - Otherwise we could compute a hash value and compare that instead
  - Nodes contain a **value** and a **pointer to the next node**
  - Keeping the list sorted allows us to locate elements quicker
  - We use **two sentinel nodes** (head and tail) to simplify the implementation
  - Example: Set<int> containing 25, -34, 23:



Traversing the List

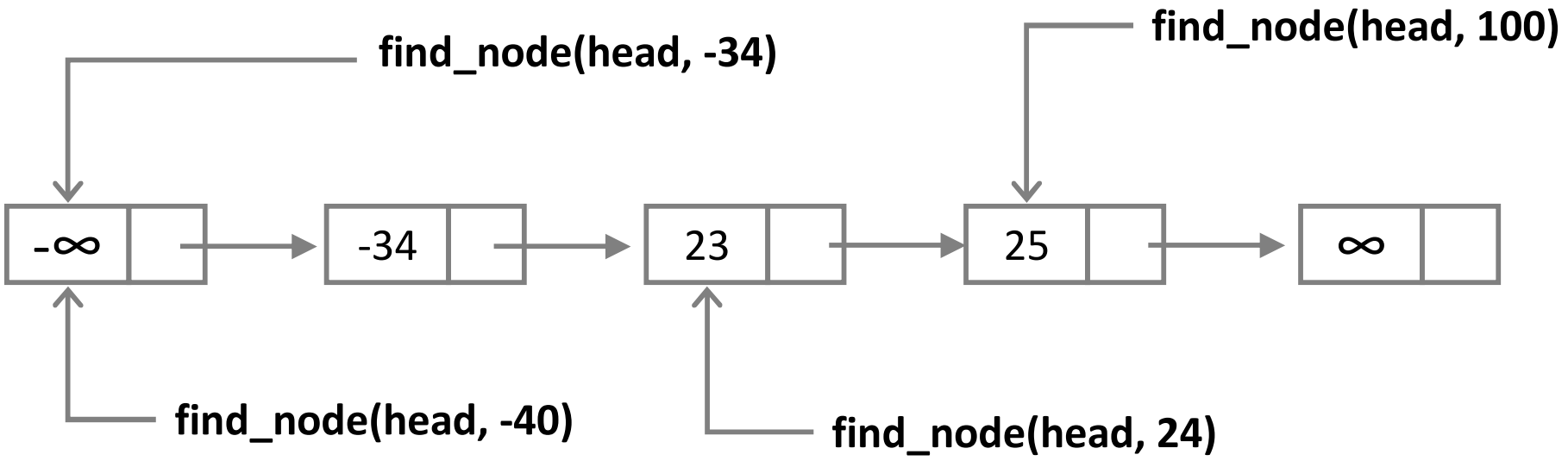
- We'll need a way to traverse the list and find the right location to modify the list for implementing our add/remove/contains methods:

```
Node* find_node(Node *start, T val);
```

- Usage: res = find_node(start, val)
  - Start traversing the list at "start", precondition: start.val < val
  - Continue traversing until you reach a node with a value >= "val", return the preceding node
  - The node returned is the largest node with a smaller value than val
- Discussion
  - The right sentinel node (tail of the list) will never be returned because it never has a value < than any other value
  - Thus, the returned value always has a valid next pointer

Examples for find_node()

- Example: Set s = {23,-34,25}



Implementing find_node

- We traverse the list starting from “start” following next pointers
  - We use a look-ahead pointer “next” to determine when we need to stop traversing
  - The returned node “curr” has a value < val
  - The following node “next” has a value >= val

```
Node* find_node(Node *start, T val) {
  Node *curr=start;
  Node *next=curr->next;
  while( next && next->val < val ) {
    curr=next;
    next=curr->next;
  }
  return curr;
}
```

A Sequential Set Implementation

- We'll implement
  - **add**(), **remove**(), **contains**() based on our find_node() helper
- The sequential version will be the basis for several concurrent versions of Set<T>
- We will explore the following schemes
  - Coarse grained locking
  - Fine grained locking
  - Optimistic
  - Lazy
- Sometimes we will have to make some changes to the data structures along the way as well

Set<T>::contains()

```
class Node {
  T val;
  Node *next;
}
```

```
class Set {
  Node *tail = new Node(∞, NULL);
  Node *head = new Node(-∞, tail);

  bool contains(T val);
  bool add(T val);
  bool remove(T val);
}
```

- Contains
  - Node “res” returned by find_node() is largest node < val
  - So check res->next to see if it’s value equals val

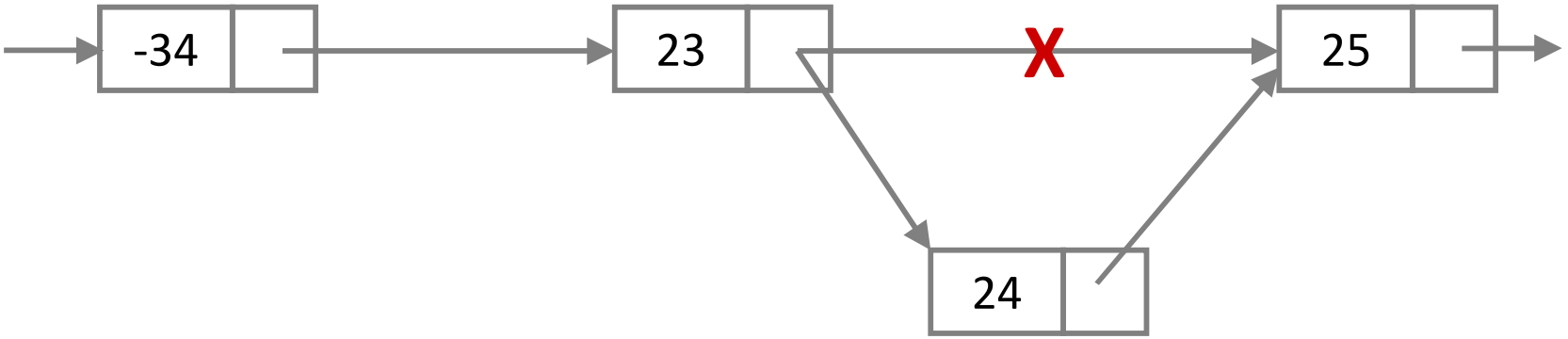
```
bool contains(T val) {
  res = find_node(head, val);
  return res->next->val == val;
}
```

Set<T>::add()

```
bool add(T val) {
  res = find_node(head, val);
  if res->next->val == val return false;

  node = new Node(val);
  node->next = res->next;
  res->next  = node;
  return true;
}
```

- Example: Set s = {-34,23,25}
  - s.add(24);

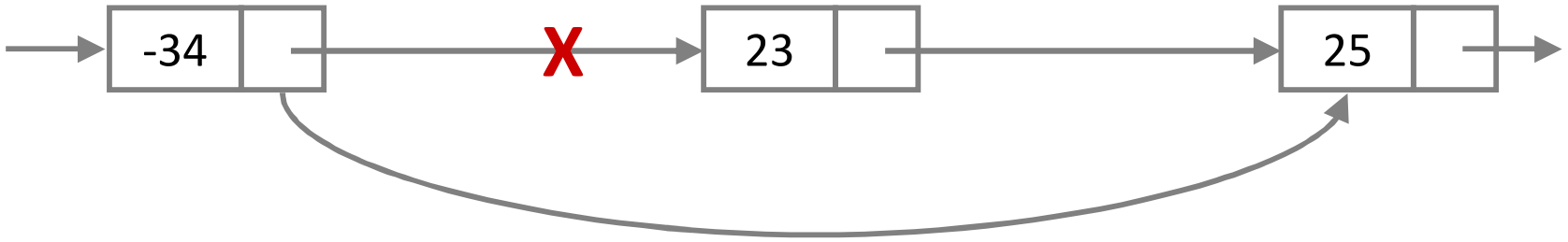


Set<T>::remove()

```
bool remove(T val) {
  res = find_node(head, val);
  if res->next->val != val return false;

  Node *node = res->next;
  res->next = node->next;
  delete(node);
  return true;
}
```

- Example: Set s = {-34,23,25}
  - s.remove(23);



Discussion (1)

- This is a simple implementation of a **Set** that obeys the following **representation invariants:**
  - Sentinel nodes are never added or removed
  - Sentinel nodes are never searched for
  - Elements are sorted by value
  - No duplicates are present
- It is easy to see that all our methods preserve these invariants
- In addition, this is the **abstraction map** that relates to the underlying concept of a Set:
  - An element is in the set if and only if it is reachable from *head*
  - Note this is true even if we don't delete the node, unlinking the node is enough

Discussion (2)

- Memory management is straightforward in this implementation
  - We use **new** and **delete**; a deleted node is not accessible by another thread after deletion
  - We'll have to be more careful about memory management with concurrent versions later: e.g., traversing nodes that may be concurrently deleted
  - This is also known as **freedom of interference:** no one interferes with our internal representation, the nodes and their links, including the lifetime of the memory is under our control
  - In garbage collected languages we don't have to worry about explicitly deleting the node, unlinking it will be enough

Course-Grained Locking (1)

- Course-grained locking means protecting each object with a lock
  - In Java this could be accomplished with the *synchronized* keyword
  - In OpenMP we could enclose the body of each method in a *critical section*
  - In C++ we can use RAII with *lock guards*, so we don't forget to release locks

```
class Set {
  Node *tail = new Node(∞, NULL);
  Node *head = new Node(-∞, tail);
  Lock lock;
}
```

```
Set::contains/add/remove(T val) :
  lock.acquire();
  ... // body of method
  lock.release();
```

Course-Grained Locking (2)

- Reader-Writer locks may be used
  - Several contains() calls can safely proceed concurrently
  - If contains() calls are much more common than add/remove this might be a significant performance improvement

```
class Set {
  Node *tail = new Node(∞, NULL);
  Node *head = new Node(-∞, tail);
  Lock rwlock;
}
```

```
Set::contains(T val) :
  rwlock.reader_acquire();
  ... // body of method
  rwlock.reader_release();
```

```
Set::add/remove(T val) :
  rwlock.writer_acquire();
  ... // body of method
  rwlock.writer_release();
```

Coarse-Grained Locking Discussion

- Our Course-grained implementation offers the same representation invariants and abstraction map as the sequential version

- Our implementation is **linearizable** and offers the same progress guarantees as the underlying lock
  - Linearization points are e.g., immediately after the lock is acquired
  - If our lock is **starvation free**, so is our Set<T> implementation

- In general
  - Coarse-grained locking works well only with **low levels of concurrency**
  - Otherwise the concurrent object may become a bottleneck
  - Advantage is the relative **implementation simplicity**

Fine-Grained Locking

- Fine-grained locking means that we don't lock access to the whole object, but only parts of it
  - Like in our example with Set, we often have nodes (e.g., stacks, lists, trees) linked by references / pointers
  - We can **embed a lock in each node** and protect concurrent updates by locking the affected node's locks

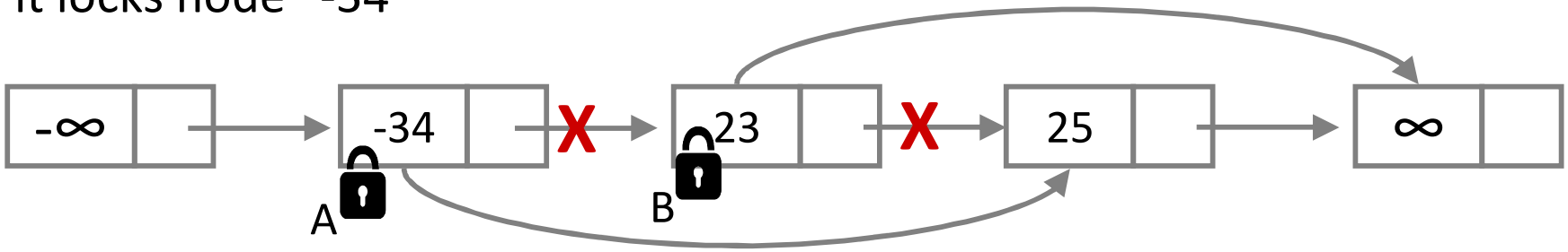
```
class Node {
  T val;
  Node *next;
  Lock lock;

acquire() : // lock the node
  lock.acquire();

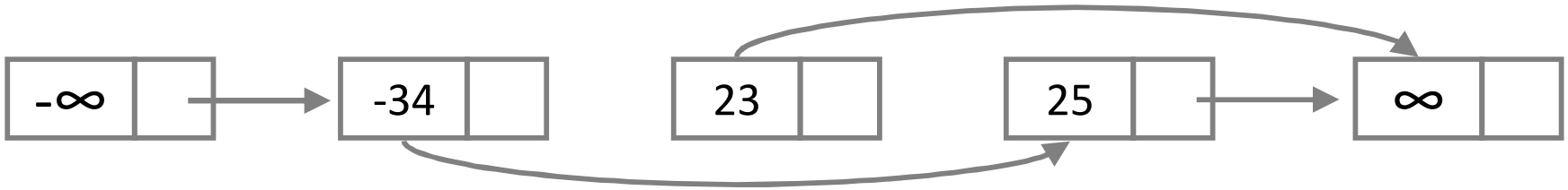
release() : // unlock the node
  lock.release();
}
```

Single-Node Locking

- Is locking just one node enough?
  - Example set S = {-34, 23, 25}
  - If thread A wants to **remove 23** it needs to modify the next pointer of -34, so it locks node “-34”

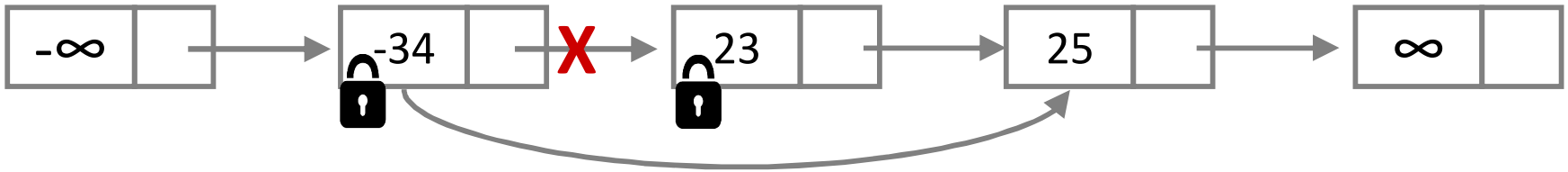


  - If thread B wants to concurrently remove 25, it needs to modify the next pointer of 23, so it locks node “23”
  - The result of this concurrent execution scenario is that **23 will be removed**, but **25 will still be part of the set** (because thread B modified the next pointer in a node that was concurrently removed).



Discussion

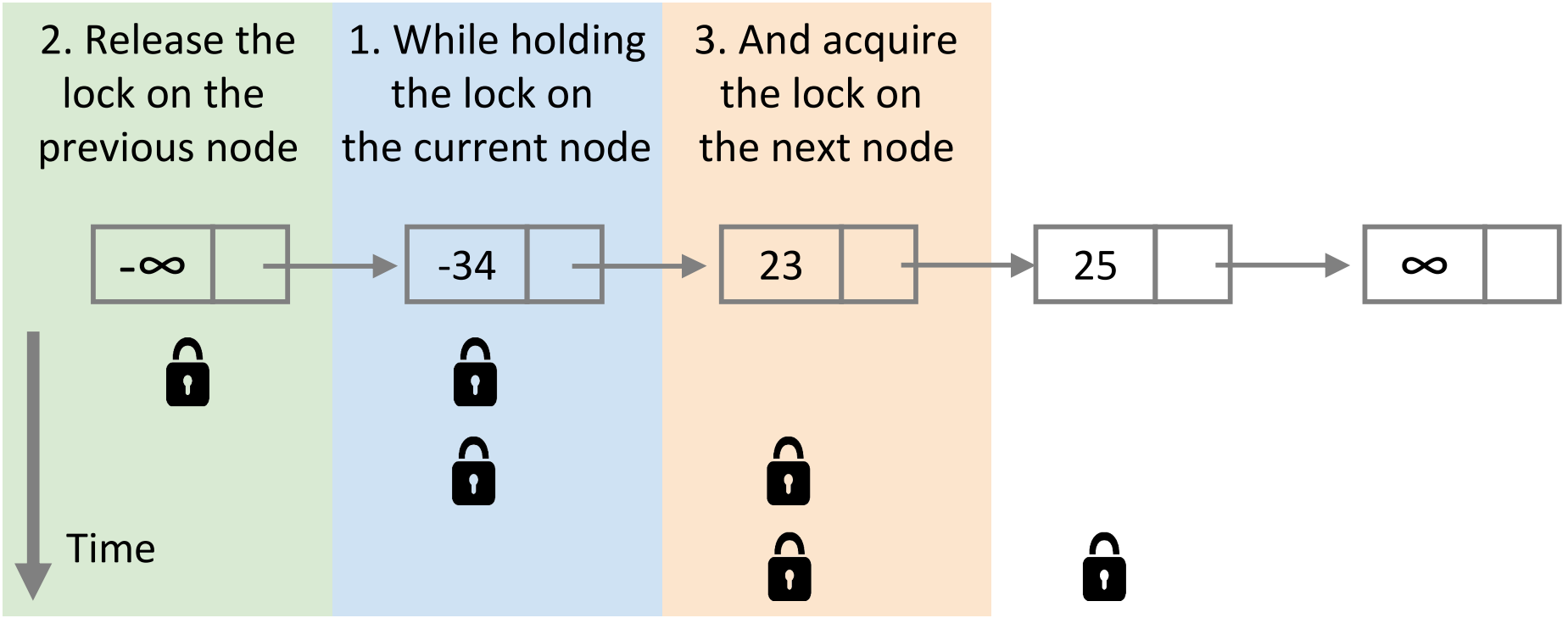
- Locking a single node is **not** enough, we need to **lock two nodes**
  - The node that is about to be modified and the next, to-be-removed node
  - For example: For removing “23” we need to lock “-34” and “23”



- Similarly, locking only one node at a time is not enough while **traversing** in the presence of concurrent removals
  - Example: between unlocking one node and locking the next node the node could have been removed by another thread

- We can solve this problem by locking nodes in find_node() in a **hand-over-hand locking scheme**

Hand-over-Hand locking



- HoH Locking
  - Also called **lock coupling**
  - Allows concurrent modifications on distinct parts of the list
  - But threads can never overtake each other while they traverse the list

Implementing find_node() with HoH Locking

```
Node* find_node_hoh(Node *start, T val) {
  Node *curr=start;
  curr->acquire();
  Node *next=curr->next;
  next->acquire();
  while( next && next->val < val )
  {
    curr->release();
    curr=next;
    next=curr->next;
    next->acquire();
  }
  return curr;
}
```

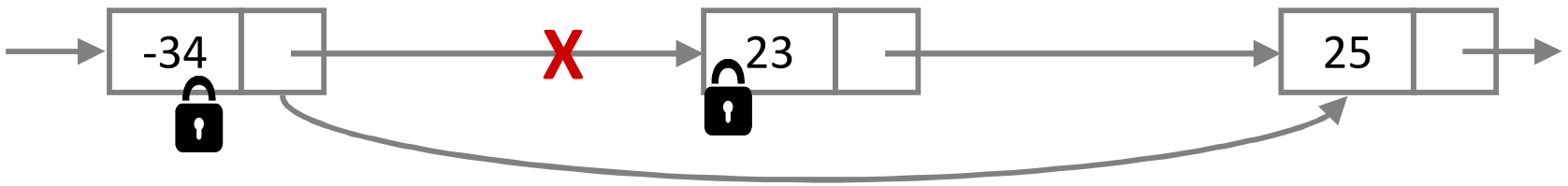
- Upon return
  - The returned node "curr" and the following node (curr->next) will be locked
  - So we have to be careful that the caller unlocks these two nodes!

Set<T>::remove()

```
bool remove(T val) {
  res = find_node_hoh(head, val);
  if res->next->val != val
    res->unlock(); res->next->unlock();
    return false;

  Node *node = res->next;
  res->next = node->next;
  res->unlock(); node->unlock();
  delete(node);
  return true;
}
```

- Example: Set s = {-34,23,25}
  - s.remove(23);



Fine-Grained Locking Discussion (1)

- add() and contains() are implemented similarly
  - Any time we access a “next” pointer, we do so under protection of a lock
  - Disadvantage of this version is that we need to acquire locks even for contains()

- Q: Do we really need to acquire locks when we call contains()?
- A: Yes, otherwise we’d have a data race, some threads may change the next pointer of nodes, while we traverse them

- What about using atomic operations for accessing the *next* pointer?
  - If we use atomic load/store operations we can safely traverse the list without locking
  - This leads to the **Optimistic Locking** strategy

Fine-Grained Locking Discussion (2)

- The fine-grained locking implementation is also **linearizable** and offers the same progress guarantees as the underlying lock
  - If our lock is **starvation free**, so is our Set<T> implementation
  - There is **no deadlock**, because we always acquire locks in the same order
  - If different methods would use different acquisition order; e.g., one from head, one from tail there could be **deadlock**
  - Linearization points: when find_node_hoh() returns with two locks acquired

Optimistic Locking

- Our fine-grained locking scheme has some problems
  - Threads traversing the list cannot overtake each other
  - contains() has to use locks, even though it does not modify the set

- Idea of **Optimistic Locking**:
  - Traverse the list without locks
  - Then lock the nodes found and validate that they are still part of the set
  - Then perform the operation and unlock

```
class Node {
  T val;
  atomic Node *next; // all operations are atomic
  Lock lock;
}
```

Set<T>::remove()

```
bool remove(T val) {
  while(true) {  // retry if validation fails
    res = find_node(head, val);     // traverse ignoring locks
    res->acquire(); res->next->acquire();  // lock the nodes
    if( validate(res, res->next) ) {
      if res->next->val != val
        res->release(); res->next->release();
        return false;

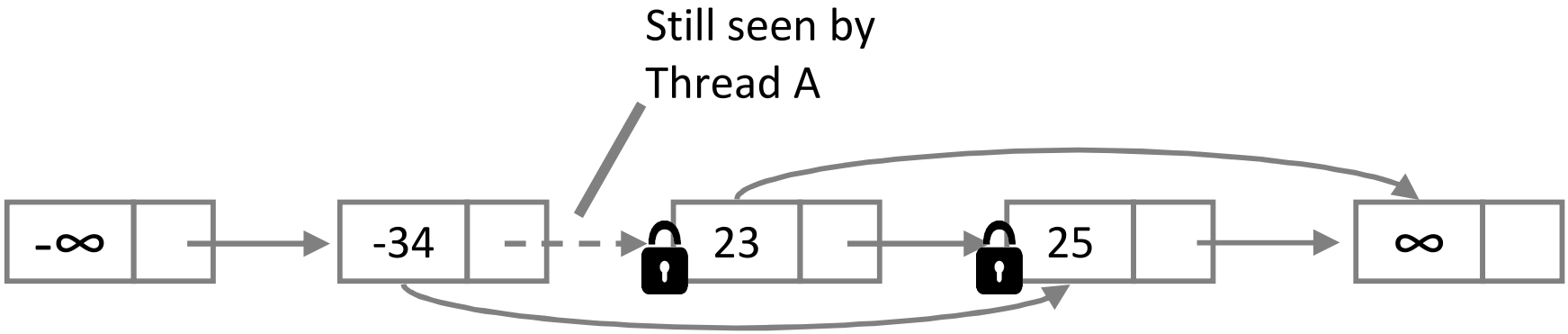
      Node *node = res->next;
      res->next = node->next;
      res->release(); node->release();
      "delete"(node); // must be reference counted!
      return true;
    }
    res->release(); node->release();
  }
}
```

Optimistic Locking Discussion (1)

- add() and contains() are implemented similarly
- Initially we traverse the list while ignoring locks
  - Thus the list may change while we traverse it, nodes may be added or removed
  - But all updates are atomic, so when we follow a next pointer, it will always point to a node that can be traversed
  - However, we need a validation step after locking nodes to validate that the found nodes are still part of the Set

```
bool validate(Node *curr, Node *next) {
  Node *node=head;
  while( node && node->val <= curr.val ) {
    if( node==curr )
       return node->next==next;
    node = node->next;
  }
  return false;
}
```

Why Validation is Required



- In this situation node “23” was removed from the set, but a concurrent add(24) executed by thread A may still have seen the link from -34 to 23 while traversing the list
  - Thread A then locks nodes “23” and “25”
  - Thread A then tries to validate nodes “23” and “25”: I.e., is node “23” still reachable from head and does “23” point to “25”?
  - This validation fails and the whole add(24) operation has to retry by traversing the list without locks

Optimistic Locking Discussion (2)

- The previous example also shows a challenge with memory management
  - Node “23” was removed but it was still reachable by other threads
  - If we have a language with automatic garbage collection, problem is solved
  - If not, we need other solutions, e.g., manual reference counting, using C++ shared_ptr, etc.

- The optimistic locking implementation is also **linearizable**
  - Linearization points are the same as in the fine-grained locking implementation

- The optimistic locking strategy is **no longer starvation free**
  - Even when the underlying lock is starvation free!
  - Validation may fail again and again, if the list is concurrently modified by other threads

Lazy Locking

- With optimistic locking, each validation requires that we traverse the entire list to check whether nodes are still reachable
  - We can avoid this overhead by simply **marking** removed elements as such

- Thus, we separate node removal in two parts:
  - **First, logically remove** the node by marking it (set a bit)
  - **Second, physically remove** the node (switch pointers, so that the node is no longer accessible and delete the node)

```
class Node {
  T val;
  atomic Node *next; // all operations on next are atomic
  atomic bool removed=false; // is the node logically removed?
  Lock lock;
}
```

Lazy Locking Validation

- This version has a **new abstraction map**:
  - An element is in the set if and only if it is reachable from head and the "removed" flag is not set

```
bool validate(Node *curr, Node *next) {
  return !(curr->removed) &&
         !(next->removed) &&
         curr->next == next;
}
```

- add() and remove() are similar to the optimistic locking case
  - They simply use the new validation procedure
  - Of course remove() also needs to set the "removed" flag
  - It is critical that his happens **before** the node is unlinked from the list

Lazy Locking contains()

```
bool contains(T val) {
  res = find_node(head, val);
  return  res->next->val == val &&
          !res->next->removed;
}
```

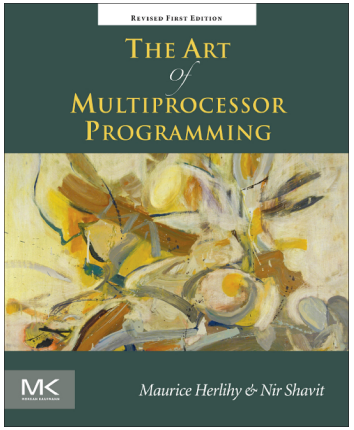
- In the Lazy Locking scheme
  - contains() is wait-free
  - add() and remove() traverse the list only once
- This implementation is linearizable, but determining the linearization points is more challenging – lin. points depend on the execution history

Summary

- We have seen several, progressively more complex, examples of lock-based concurrent data structures

- For all but the simple coarse-grained locking scheme we have to carefully consider several issues:
  - Which parts of the data structure do we need to lock to ensure correct operation?
  - Can deadlock occur?
  - Is accessing data outside of the protection of locks safe? Are we using atomic operations when needed?
  - Memory management: Can threads still access removed parts of the data structure? How do we deal with this situation?

Further Reading



- Maurice Herlihy, Nir Shavit *"The Art of Multiprocessor Programming"*
  - Chapter 9 “Linked Lists: The Role of Locking”